

Lab 03 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can check at the terminal.

Question 1 — Three behaviours, one rule

Answer. The rule: **a container lives exactly as long as its PID 1**. `alpine`'s default command is `/bin/sh`; with no standard input, the shell reads end-of-file and exits at once → the container exits. `nginx` stays in the foreground and never returns → the container lives, and blocks your terminal because you did not say `-d`. `-it alpine sh` gives the shell an open input and a terminal → it waits for your commands, so the container lives until you type `exit`.

Why. The engine only launches a process inside namespaces and waits for it to end. There is no notion of a "service": what keeps the container alive is a process that does not end.

Nuance. `podman run nginx` without `-d` does not mean `nginx` runs "differently": it is identical, only your terminal is attached to its outputs. `Ctrl+C` then sends `SIGINT` to PID 1 and stops it.

Example.

```
podman run alpine;                podman ps -a -l --format '{{.Status}}'    # Exited (0)
podman run -d nginx:alpine;        podman ps -l --format '{{.Status}}'        # Up
podman run -it alpine sh -c 'echo "I live as long as you want"; exit 7'; echo $?    # 7
```

Question 2 — The `&` that kills

Answer. The script is PID 1. `java ... &` starts Java in the background and returns immediately; `echo` runs; the script reaches its end and exits with code `0`. PID 1 dead, the kernel kills everything else in the namespace, Java included. Fix: launch Java in the foreground **and** as the last line, with `exec`:

```
#!/bin/sh
echo "API started"
exec java -jar /app/api.jar
```

Why. `exec` replaces the shell with Java, which becomes PID 1: it lives as long as it wants and receives `SIGTERM` directly. Without `exec` but without `&`, the script would wait for Java (the container would live) but would remain PID 1 in front of it — and would not forward `SIGTERM` (question 3 of lab 04).

Nuance. Code `0` is misleading: everything "went well" from the script's point of view. It is an example of a container that fails without an error — `on-failure restart policies` would not even restart it.

Example.

```
podman run --rm -v "$PWD":/s alpine /s/demarrage-casse.sh    # returns at once
podman run -d --name ok -v "$PWD":/s alpine /s/demarrage-correct.sh && podman top ok    # sleep
as PID 1
```

Question 3 — Ten seconds and 137

Answer. `sleep` is PID 1, and the Linux kernel makes PID 1 ignore any signal for which it installed no handler. `sleep` installs none: `SIGTERM` is ignored. Podman waits the grace period (10 s), announces it is moving to `SIGKILL` — which nobody can ignore — and the process dies killed: code `128 + 9 = 137`. `143` (`128 + 15`) only appears when `SIGTERM` actually terminated the process.

Why. That PID 1 protection exists so that a clumsy `kill -TERM 1` does not bring down a whole machine. Inside a container, it turns against you.

Nuance. This is not specific to `sleep`: any program without a `SIGTERM` handler behaves like this as PID 1 — including a shell script, or a `java` launched behind a shell. The warning Podman prints (`resorting to SIGKILL`) is precious: Docker kills silently.

Example.

```
podman run --rm alpine sh -c 'kill -TERM 1; echo survived'      # "survived": PID 1 ignored its
own TERM
podman run -d --name v alpine sleep 300; time podman stop v    # 10 s, code 137
```

Question 4 — What `--init` changes

Answer. `--init` inserts `podman-init` (a binary of a few KB, `catatonit`) as PID 1; `sleep` becomes its child, PID 2. `podman-init` knows two things: forward signals to its child and reap zombies. At `stop`, it receives `SIGTERM` and passes it to `sleep`, which — no longer PID 1 — suffers the default action: dying. Code `143`, immediately. `podman exec idle ps` shows `1 podman-init` then `2 sleep`.

Why. The kernel protection only applies to PID 1. By moving your program to PID 2, it gets normal signal behaviour back.

Nuance. `--init` is a band-aid: it does not make your application capable of a clean shutdown, it only makes it *cleanly killable*. A Spring Boot API handles `SIGTERM` itself; it does not need `--init`, it needs to **receive** it (exec form). `--init` remains useful for images that launch several processes and produce zombies.

Example.

```
podman run --rm --init alpine ps -o pid,comm                  # 1 podman-init, 2 ps
```

Question 5 — `-i` without `-t`, `-t` without `-i`

Answer. `-i` keeps `stdin` open and connected to your keyboard; `-t` allocates a pseudo-terminal (prompt, echo, key handling). `podman run -t alpine sh`: you see a prompt, but `stdin` is not connected — your keystrokes go nowhere, `ls` does nothing, and the container stays stuck there until you kill it from another terminal (`podman rm -f -t 0`). `podman run -i alpine sh`: no prompt or echo, but what you type is transmitted: `ls` runs and prints its result, without comfort.

Why. They are two independent channels: `-i` concerns the data flow, `-t` the presentation. A shell only needs `-i` to work; it needs `-t` to be pleasant.

Nuance. `-i` alone is the form for scripts: `echo "SELECT 1" | podman exec -i db psql -U app` works, whereas with `-t` it would fail (`the input device is not a TTY`). A classic CI bug.

Example.

```
echo 'echo "got: $((6*7))"' | podman run -i --rm alpine sh      # got: 42 — no prompt
podman run -it --rm alpine sh                                # prompt "/ #", Ctrl+D to leave
```

Question 6 — `attach` and `Ctrl+C`

Answer. `attach` hooked their terminal onto the streams of **PID 1** — the API itself. `Ctrl+C` sent `SIGINT` to that process, which stopped; the container died with it. The two right ways: `podman logs -f my-api` (reads the logs captured by `common`, `Ctrl+C` only stops the display) or `podman exec -it my-api sh` (new process, no effect on PID 1).

Why. `attach` creates nothing: it reconnects your terminal to the existing pipes of the main process, signals included. It is exactly what you would have by running the container in the foreground.

Nuance. There is an escape: `Ctrl+P` `Ctrl+Q` detaches without stopping (if the container was started with `-it`), and `podman attach --sig-proxy=false` prevents signal forwarding. But the real answer is not to use `attach` to read logs.

Example.

```
podman logs -f --tail 20 my-api      # Ctrl+C: the container keeps running
podman attach --sig-proxy=false my-api # Ctrl+C will not be forwarded
```

Question 7 — 137, 143, 127

Answer. `api` (137): killed by `SIGKILL` — either a `stop` whose grace period expired, or the OOM killer. Confirm: `podman inspect --format '{{.State.OOMKilled}}' api`, then `podman events --since 1h | grep api` to see whether there was a `stop`. `worker` (143): received `SIGTERM` and terminated — a voluntary stop (deployment, `podman stop`); confirm with `podman events` or `journalctl`. `batch` (127): the command was not found — the application never started (image or `CMD` error). Confirm: `podman logs batch` (message `executable file not found`) and `podman inspect --format '{{json .Config.Cmd}}' batch`.

Why. Above 128, the code is `128 + signal number`. Below, it is the code chosen by the program — or by the shell/runtime when the program could not be launched.

Nuance. A 137 with `OOMKilled: false` and no `stop` in the events may come from a manual `kill -9` or an orchestrator. And `worker`'s 143 at the same time as `api`'s 137 suggests a grouped stop where `api` failed to stop cleanly: the symptom of a shell in front (lab 04).

Example.

```
podman inspect --format 'oom={{.State.OOMKilled}} finished={{.State.FinishedAt}}' api
podman events --since 1h --filter container=api
```

Question 8 — Logs in a file

Answer. `podman logs` only returns what `common` captured on `stdout` / `stderr` of PID 1. By writing to a file, the application bypasses that channel: nothing is captured. Mounting the folder on the host makes the file readable, but remains a bad answer: the logs escape the tooling (`podman`

logs, journald, collection agents), every container invents its own path, rotation is not handled, and a removed container leaves orphan files.

Why. The container model treats logs as a **stream**: the engine captures it, the tooling routes it (file, journal, Loki, Elastic). A file inside the container is local state, contrary to the disposable nature of the container.

Nuance. Spring Boot logs to the console by default: it is enough **not** to define `logging.file.name`. If a file format is imposed, the solution is a *sidecar* or an agent that reads the stream, not a mount.

Example.

```
podman run -d --name l alpine sh -c 'echo visible; echo invisible > /tmp/app.log; sleep 100'
podman logs l                                # visible
podman exec l cat /tmp/app.log                # invisible – only by entering
```

Question 9 — stop / start versus rm / run

Answer. After `stop` then `start`: the data is **kept** — the container's writable layer still exists, PostgreSQL finds its files again. After `rm` then a new `run`: the data is **lost** — `rm` destroyed the writable layer, and the new container starts from the image.

Why. `stop` only acts on the process; the container (configuration + layer) remains. `rm` removes the container object, layer included.

Nuance. The `postgres` image declares a `VOLUME`: the data goes into an anonymous volume that survives the `rm` but is attached to nothing any more — unrecoverable in practice. The named volume (lab 06) is the only real persistence.

Example.

```
podman run -d --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman exec db psql -U postgres -c 'create table t(x int)'
podman stop db && podman start db && podman exec db psql -U postgres -c '\dt'    # t is there
podman rm -f -t 0 db && podman run -d --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman exec db psql -U postgres -c '\dt'                                       # nothing left
```

Question 10 — --restart=always and the reboot

Answer. Under Docker, the daemon re-reads the policies at start-up and relaunches the containers. Under Podman there is no daemon: `--restart=always` is enforced by `common` as long as the container *exists in a live session*, but after a reboot nothing runs to re-read the policy. The Podman way: a **Quadlet** file (`/etc/containers/systemd/api.container`, or `~/config/containers/systemd/` rootless) describing the container, and `systemctl enable --now api` — `systemd` starts it at boot and restarts it on failure.

Why. Podman chose not to reinvent a service manager: Linux has one, `systemd`, with its dependencies, its logs and its start at boot. A Podman *restart policy* only covers the life of a session.

Nuance. Rootless, you additionally need `loginctl enable-linger <user>` for the user's services to start without an open session. On a WSL workstation, none of this is usually necessary: development containers need not survive a reboot.

Example.

```
# ~/.config/containers/systemd/api.container
[Container]
Image=registry.internal/myapp/api:1.4.2
PublishPort=8080:8080
[Install]
WantedBy=default.target
```

```
systemctl --user daemon-reload && systemctl --user start api && systemctl --user status api
```

Question 11 — The logs of the first attempt

Answer. In `podman logs <container>`: logs **accumulate** on the same container at every restart, the first attempt is at the top. `podman restart` does not erase them either — but you lose `.State.ExitCode` and `.State.FinishedAt` of the last run, and above all the container goes back into its loop. Look first.

Why. An automatic restart relaunches the **same** container (same ID, same writable layer, same log file), it does not create a new one. `podman events` additionally gives the exact chronology (died, restart).

Nuance. A `podman rm` (or `--rm`) removes everything, logs included. And a container restarting in a loop can produce huge logs: `--tail` and `--since` are your friends.

Example.

```
podman logs --timestamps unstable | head -20      # the first run
podman events --since 10m --filter container=unstable
```

Question 12 — From least to most intrusive

Answer. (1) `podman inspect`: reads metadata, no effect — configuration, state, OOM, host PID. (2) `podman logs`: reads what `common` already captured — what the application says about itself. (3) `podman stats`: reads the cgroups — real CPU, memory, I/O, without touching the container. (4) `podman top`: runs a `ps` on the host side over the container's PIDs — which process consumes, which threads. (5) `podman exec`: creates a process **inside** the container — the most intrusive, but the only one allowing a `jstack` or a `curl localhost:8080/actuator`.

Why. The first four observe from outside, through the engine or the kernel; only `exec` modifies the inside (one more process, resources consumed inside the container's cgroup).

Nuance. With the host PID given by `inspect`, you can go further without `exec`: `cat /proc/<pid>/status`, `strace -p <pid>` — since rootless the container is a process of your user. And a *distroless* image has no shell: `exec` is not possible there (lab 05).

Example.

```
podman stats --no-stream api
podman top api pid,pcpu,comm
podman exec api jcmd 1 Thread.print | head -50
```

Question 13 — API and database in the same container

Answer. Three consequences: (1) **a single PID 1**: you need a supervisor (`supervisord`) to hold two processes, and if the database dies the container does not know it — or conversely, the API dies and takes the database with it; (2) **coupled life cycle**: redeploying the API forces a PostgreSQL restart, with its connections and its cache; (3) **resources and observability mixed up**: one memory limit, one mixed log stream, impossible to scale the API without duplicating the database.

Why. The container is designed around *one* main process whose life is the container's. Two processes means two life cycles in an object that has only one.

Nuance. Podman has an object for "several containers that must live together": the **pod** (`podman pod create`), which shares network and life cycle while keeping one container per process — the same concept as Kubernetes. That is the correct answer to the "start simply" need.

Example.

```
podman pod create --name stack -p 8080:8080
podman run -d --pod stack --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman run -d --pod stack --name api my-api:1.0      # reaches db on localhost:5432
```

Question 14 — `--rm` and production

Answer. For a one-off command, `--rm` avoids piling up corpses. For a production service, it destroys on exit exactly what you need after an incident: the **logs**, the **exit code**, the **writable layer** (temporary files, *heap dump*), and the possibility of `podman inspect`. The container is dead and there is nothing left to examine. The combination with `--restart` is contradictory by construction: `--rm` removes the container when it exits, `--restart` wants to relaunch it when it exits — you cannot relaunch what you just erased. Podman refuses it explicitly.

Why. The `Exited` container is the object of the autopsy. A service that crashed at 3 a.m. must be inspectable at 9 a.m.

Nuance. Orchestrators (Kubernetes, Compose) handle the removal of finished containers themselves, with a delay and log retention. `--rm` remains perfect for tool containers: compilation, database migration, interactive `psql`.

Example.

```
podman run --rm -d --restart=always nginx:alpine
# Error: the --rm option conflicts with --restart, when the restartPolicy is not "" and "no"
```